

Assignment 1

Name: Excellence Oseagwina Oseobulu

Matric Number: COSC/22404

Course: CSC 423 (ARTIFICIAL INTELLIGENCE)

Project on:

1. Spam Classifier to classify email/SMS text as spam or not a spam.
2. House Price Regression Model to predict house prices from housing features.

It includes data loading, pre-processing, model training, metrics, charts, explanations, and reusable prediction functions.

Setup, Step by Step process and Requirements

Import the required Python libraries for data handling, plotting, machine learning, and model evaluation.

Code

```
import io
import zipfile
import urllib.request
import warnings

import numpy as
np import pandas
as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LinearRegression,
LogisticRegression from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    mean_absolute_error,
    ,
    mean_squared_error,
    precision_score,
    r2_score,
    recall_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC

warnings.filterwarnings("ignore")

plt.style.use("seaborn-v0_8-whitegrid")
sns.set_palette("Set2")
RANDOM_STATE = 42

print("Libraries loaded successfully.")
```

Output

Libraries loaded successfully.

1: Build a Spam Classifier

Question

Build a text classifier that receives an email or SMS message and predicts whether it is spam or not spam.

Answer

1. Load messages as text.
2. Convert text to numbers using CountVectorizer.
3. Train Logistic Regression and SVM models.
4. Compare Precision, Recall, and Accuracy.
5. Create an `is_spam(email_text)` function that returns True or False.

Code

```
def load_sms_spam_dataset():
    # Load the public UCI SMS Spam Collection dataset, with a small fallback.
    url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00228/smsspamcollection.zip"
    try:
        with urllib.request.urlopen(url, timeout=20) as response:
            raw_zip = response.read()
        with zipfile.ZipFile(io.BytesIO(raw_zip)) as zf:
            with zf.open("SMSSpamCollection") as f:
                data = pd.read_csv(f, sep="\t", header=None, names=["label", "message"])
            source = "UCI SMS Spam Collection public dataset"
    except Exception as exc:
        print(f"Could not download UCI dataset ({exc}). Using embedded sample dataset.")
    rows = [
        ("ham", "Are we still meeting for the group project today?"),
        ("ham", "Please send me the lecture notes when you get home."),
        ("ham", "I will call you after class."),
        ("ham", "Can you help me check my assignment before submission?"),
        ("ham", "The meeting has been moved to 4 pm."),
        ("ham", "Don't forget to bring your laptop tomorrow."),
        ("ham", "Thank you for the update."),
        ("ham", "I am on my way to campus now."),
        ("ham", "Your appointment is confirmed for Monday morning."),
        ("ham", "Let's revise linear regression tonight."),
        ("spam", "Congratulations you have won a free cash prize claim now"),
        ("spam", "URGENT your account has won money click this link"),
        ("spam", "Win a brand new phone by texting WIN to 12345"),
        ("spam", "You have been selected for a free vacation offer"),
        ("spam", "Claim your reward now limited time only"),
        ("spam", "Free entry in a weekly competition text YES now"),
        ("spam", "Lowest loan rates available apply immediately"),
        ("spam", "Exclusive deal click here to claim your bonus"),
        ("spam", "You won lottery cash send
```

```

        details to collect"), ("spam", "Act now to receive free
        airtime and gifts"),
    ]
    data = pd.DataFrame(rows, columns=["label",
    "message"]) source = "embedded fallback dataset"

    data["spam"] = (data["label"] ==
    "spam").astype(int) return data, source

```

```

spam_df, spam_source =
load_sms_spam_dataset() print(f'Dataset
source: {spam_source}') print(f'Rows:
{len(spam_df):,}') spam_df.head()

```

Output

Dataset source: UCI SMS Spam Collection public
dataset Rows: 5,572

Output

	label	message	spam
0	ham	Go until jurong point, crazy.. Available only ...	0
1	ham	Ok lar... Joking wif u oni...	0
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	1
3	ham	U dun say so early hor... U c already then say...	0
4	ham	Nah I don't think he goes to usf, he lives aro...	0

Data Exploration

The target label is spam, where:

- 1 means spam
- 0 means not spam/ham

Code

```

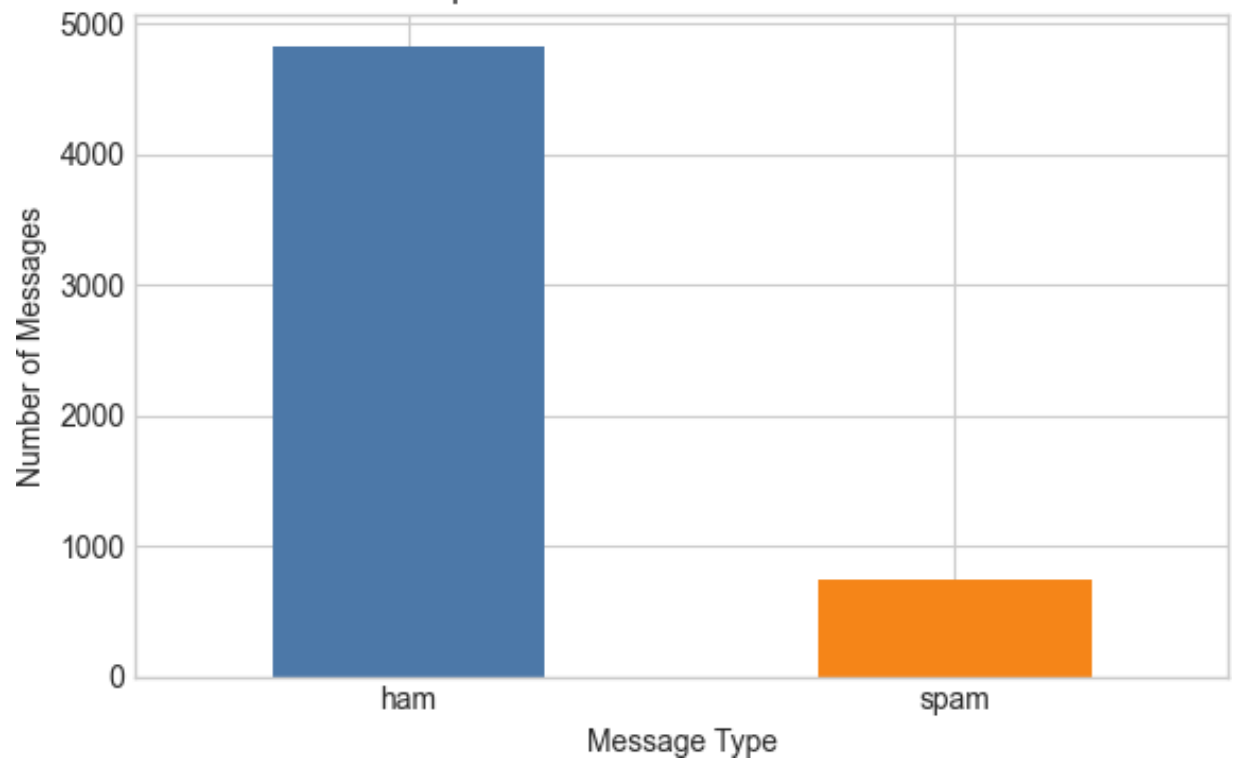
label_counts = spam_df["label"].value_counts()

fig, ax = plt.subplots(figsize=(6, 4))
label_counts.plot(kind="bar", ax=ax, color=["#4C78A8",
"#F58518"]) ax.set_title("Spam Dataset Class Distribution")
ax.set_xlabel("Message Type")
ax.set_ylabel("Number of Messages")
ax.tick_params(axis="x", rotation=0)
plt.tight_layout()
plt.show()

spam_df["message_length"] = spam_df["message"].str.len()
spam_df.groupby("label")["message_length"].describe().round(2)

```

Spam Dataset Class Distribution



Output

	count	mean	std	min	25%	50%	75%	max
label								
ham	4825.0	71.48	58.44	2.0	33.0	52.0	93.0	910.0
spam	747.0	138.67	28.87	13.0	133.0	149.0	157.0	223.0

Convert Text to Numbers

Machine-learning models cannot directly understand raw words. CountVectorizer converts each message into numeric word-count features.

Code

```
X_text = spam_df["message"]
y_spam = spam_df["spam"]

stratify_target = y_spam if y_spam.value_counts().min() >= 2 else
None
X_train_text, X_test_text, y_train_spam, y_test_spam =
train_test_split(
    X_text,
    y_spam,
    test_size=0.25,
    random_state=RANDOM_
    STATE,
    stratify=stratify_target,

)

vectorizer_preview = CountVectorizer(stop_words="english",
min_df=1)
X_train_counts =
vectorizer_preview.fit_transform(X_train_text)

print(f"Training messages: {len(X_train_text):,}")
print(f"Testing messages: {len(X_test_text):,}")
print(f"Vocabulary size: {len(vectorizer_preview.vocabulary_):,}")
print(f"Numeric matrix shape: {X_train_counts.shape}")
```

Output

```
Training messages: 4,179
Testing messages: 1,393
Vocabulary size: 7,124
Numeric matrix shape: (4179, 7124)
```

Train Logistic Regression and SVM

Two classifiers are trained using the same text-vectorization process:

- Logistic Regression
- Linear Support Vector Machine (SVM)

Code

```
logistic_model = Pipeline([
    ("vectorizer", CountVectorizer(stop_words="english", min_df=1)),
    ("classifier", LogisticRegression(max_iter=1000,
    random_state=RANDOM_STATE)),
```

```

])

svm_model = Pipeline([
    ("vectorizer", CountVectorizer(stop_words="english",
                                   min_df=1)),
    ("classifier", LinearSVC(random_state=RANDOM_STATE)),
])

logistic_model.fit(X_train_text,
y_train_spam) svm_model.fit(X_train_text,
y_train_spam)

logistic_pred = logistic_model.predict(X_test_text)
svm_pred = svm_model.predict(X_test_text)

print("Models trained successfully.")

```

Output

Models trained successfully.

Compare Model Performance

Metrics:

- Precision: Of emails flagged as spam, how many were actually spam?
- Recall: Of actual spam emails, how many did the model catch?
- Accuracy: Overall, what percentage did the model classify correctly?

Code

```

def classification_metrics(y_true, y_pred,
model_name): return {
    "Model": model_name,
    "Precision": precision_score(y_true, y_pred, zero_division=0),
    "Recall": recall_score(y_true, y_pred, zero_division=0),
    "Accuracy": accuracy_score(y_true, y_pred),
}

spam_results = pd.DataFrame([
    classification_metrics(y_test_spam, logistic_pred, "Logistic Regression"),
    classification_metrics(y_test_spam, svm_pred, "Linear SVM"),
])

spam_results

```

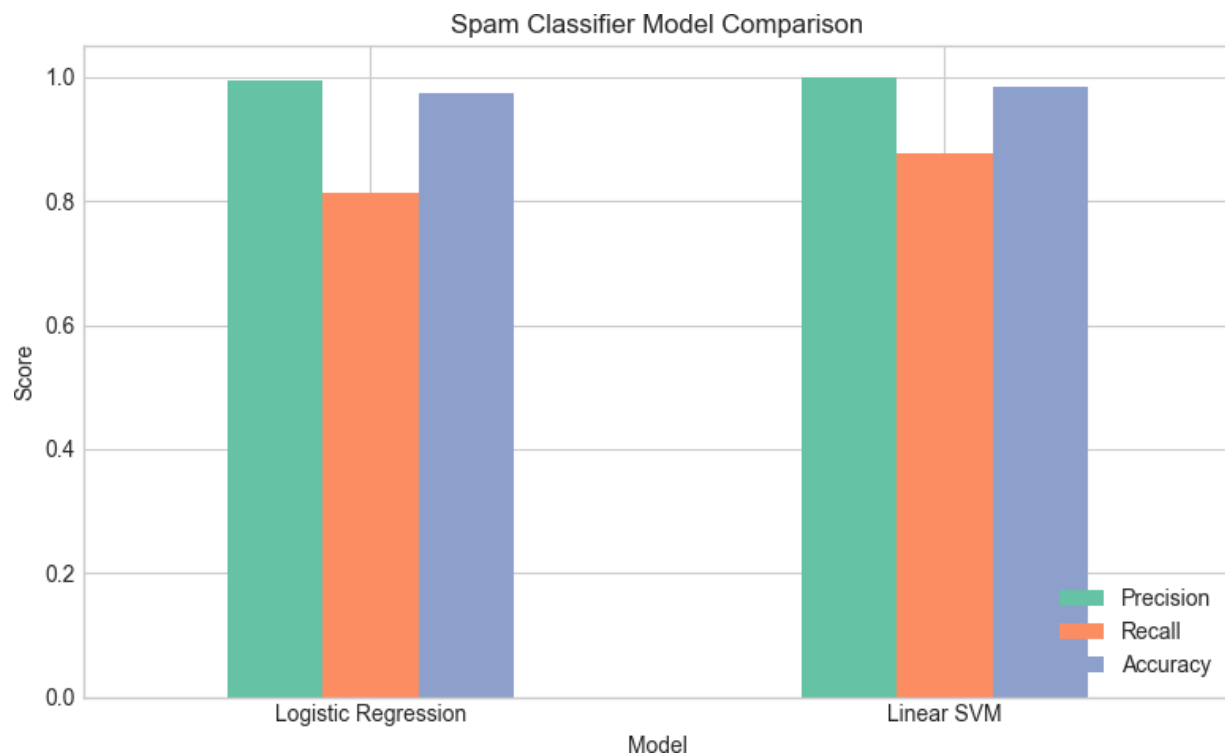
Output

Model		Precision	Recall	Accuracy
0	Logistic Regression	0.993464	0.812834	0.974156
1	Linear SVM	1.000000	0.877005	0.983489

Code

```
fig, ax = plt.subplots(figsize=(8, 5))
spam_results.set_index("Model")[["Precision", "Recall", "Accuracy"]].plot(
    kind="bar", ax=ax
)
ax.set_ylim(0, 1.05)
ax.set_title("Spam Classifier Model Comparison")
ax.set_ylabel("Score")
ax.tick_params(axis="x", rotation=0)
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()

best_spam_model_name = spam_results.sort_values(
    ["Recall", "Precision", "Accuracy"],
    ascending=False
).iloc[0]["Model"]
best_spam_model = logistic_model if best_spam_model_name == "Logistic Regression" else
svm_model
print(f"Best model selected: {best_spam_model_name}")
```



Output

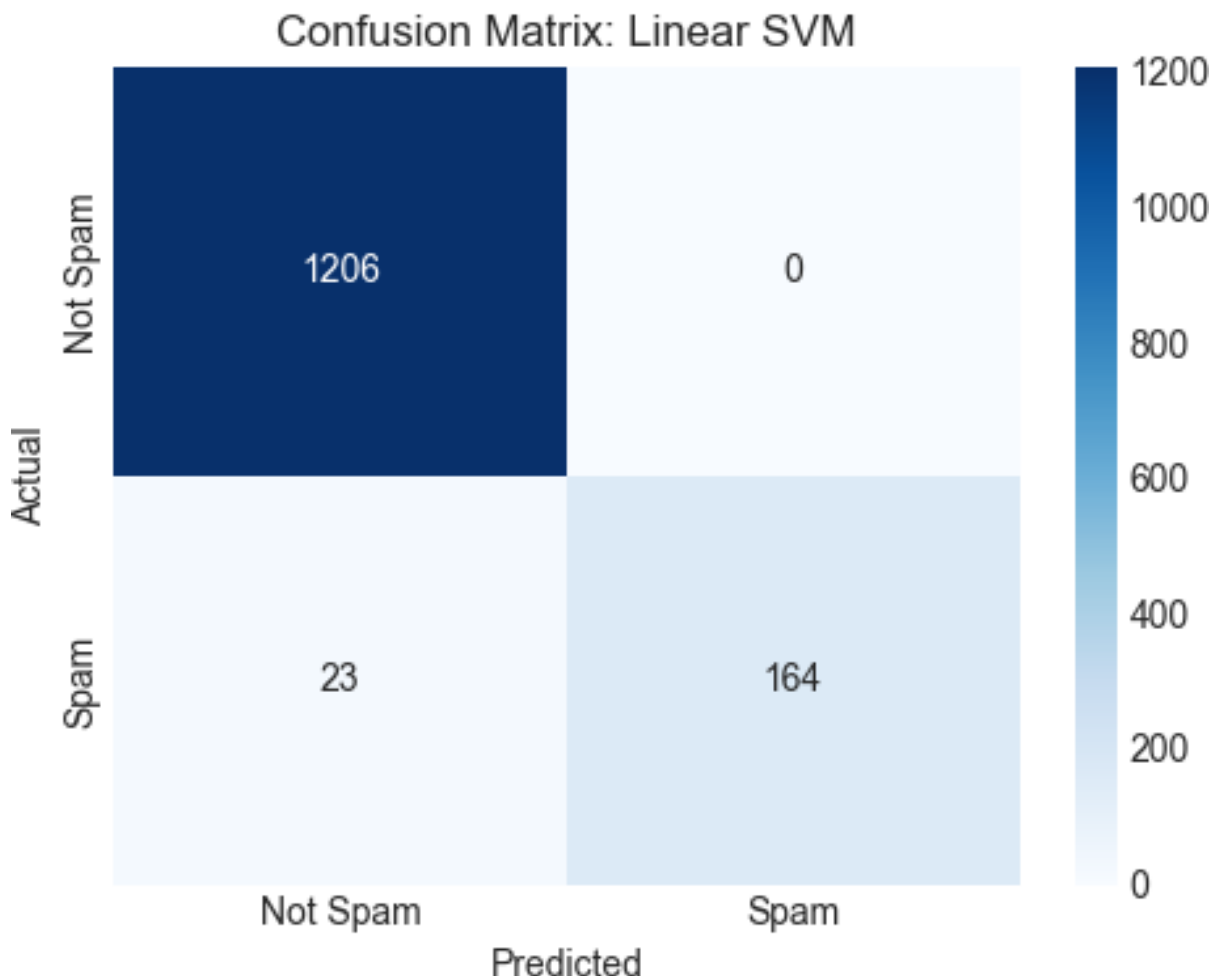
Best model selected: Linear SVM

Code

```
cm = confusion_matrix(y_test_spam, best_spam_model.predict(X_test_text))

fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Not Spam", "Spam"],
    yticklabels=["Not Spam", "Spam"],
    ax=ax,
)
ax.set_title(f"Confusion Matrix: {best_spam_model_name}")
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
plt.tight_layout()
plt.show()

print(classification_report(y_test_spam, best_spam_model.predict(X_test_text), target_names=["Not Spam", "Spam"]))
```

Output

	precision	recall	f1-score	support
Not Spam	0.98	1.00	0.99	1206
Spam	1.00	0.88	0.93	187
accuracy			0.98	1393
macro avg	0.99	0.94	0.96	1393
weighted avg	0.98	0.98	0.98	1393

Spam Prediction Function

The function below returns True when a message is predicted as spam and False otherwise.

Code

```
def is_spam(email_text):
    prediction = best_spam_model.predict([email_text])[0]
    return bool(prediction)

examples = [
    "Congratulations, you have won a free prize. Claim your cash now!",
    "Please send the assignment file before tomorrow's class.",
    "URGENT! Click this link to receive your lottery reward.",
]

for text in examples:
    print(f'{is_spam(text)} -> {text}')
```

Output

True -> Congratulations, you have won a free prize. Claim your cash now! False -> Please send the assignment file before tomorrow's class.

False -> URGENT! Click this link to receive your lottery reward.

Conclusion

The spam classifier converts message text into word-count features, trains classification models, and selects the better model using precision, recall, and accuracy. This is the same general workflow used by real email services, although production systems use much larger datasets and more advanced filters.

Part 2: My First Regression Model

Question

Predict house prices using features such as size, age, and location-related housing statistics.

Answer

1. Load a housing dataset.
2. Use Python and Pandas to inspect the data.
3. Plot the relationship between house size and price.
4. Fit `sklearn.linear_model.LinearRegression()`.
5. Make predictions on new houses.
6. Check prediction quality.

Code

```
housing = fetch_california_housing(as_frame=True)
housing_df = housing.frame.copy()

# The target is median house value in units of $100,000.
housing_df["MedHouseValueUSD"] = housing_df["MedHouseVal"]
* 100000
housing_df.head()
```

Output

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude \
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85

	Longitude	MedHouseVal	MedHouseValueUSD	
0	-122.23		4.526	452600.0
1	-122.22		3.585	358500.0
2	-122.24		3.521	352100.0
3	-122.25		3.413	341300.0
4	-122.25		3.422	342200.0

Code

```
print(f'Rows: {housing_df.shape[0]:,}')
print(f'Columns: {housing_df.shape[1]:,}')
```

```
housing_df.describe().round(2)
```

Output

Rows: 20,640
Columns: 10

Output

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	\
count	20640.00	20640.00	20640.00		20640.00		20640.00
mean	3.87	28.64	5.43	1.10	1425.48		3.07
std	1.90	12.59	2.47	0.47	1132.46		10.39
min	0.50	1.00	0.85	0.33	3.00		0.69
25%	2.56	18.00	4.44	1.01	787.00		2.43
50%	3.53	29.00	5.23	1.05	1166.00		2.82
75%	4.74	37.00	6.05	1.10	1725.00		3.28
max	15.00	52.00	141.91	34.07	35682.00		1243.33

	Latitude	Longitude	MedHouseVal	MedHouseValueUSD
count	20640.00	20640.00	20640.00	20640.00
mean	35.63	-119.57	2.07	206855.82
std	2.14	2.00	1.15	115395.62
min	32.54	-124.35	0.15	14999.00
25%	33.93	-121.80	1.20	119600.00
50%	34.26	-118.49	1.80	179700.00
75%	37.71	-118.01	2.65	264725.00
max	41.95	-114.31	5.00	500001.00

Plot House Size vs Price

The California Housing dataset uses AveRooms as a size-related feature. It is not exact square footage, but it is a useful proxy for house size in this dataset.

Code

```
plot_sample = housing_df.sample(1200, random_state=RANDOM_STATE)

fig, ax = plt.subplots(figsize=(8, 5))
sns.scatterplot(
    data=plot_sample,
    x="AveRooms",
    y="MedHouseValueUSD", alpha=0.45,
    ax=ax,
)
ax.set_xlim(0, 10)
ax.set_title("Relationship Between Average Rooms and House Price")
ax.set_xlabel("Average Rooms (Size Proxy)")
ax.set_ylabel("Median House Value (USD)")
plt.tight_layout()
plt.show()
```



Train Linear Regression Model

The model uses multiple housing features: income, house age, average rooms, average bedrooms, population, occupancy, latitude, and longitude.

Code

```
feature_cols = [
    "MedInc",
    "HouseAge"
,
    "AveRooms"
,
    "AveBedrm
s",
    "Population"
,
    "AveOccup"
, "Latitude",
    "Longitude"
]
target_col = "MedHouseValueUSD"

X_house =
housing_df[feature_cols] y_house
= housing_df[target_col]

X_train_house, X_test_house, y_train_house, y_test_house = train_test_split(
    X_house,
    y_house,
    test_size=0.2,
    random_state=RANDOM_STATE,
)

regression_model = LinearRegression()
```

```

regression_model.fit(X_train_house, y_train_house)

house_predictions = regression_model.predict(X_test_house)

print("Linear Regression model trained successfully.")

```

Output

Linear Regression model trained successfully.

Check Prediction Quality

Regression is evaluated with:

- MAE: average absolute prediction error.
- RMSE: larger errors receive stronger penalty.
- R²: how much target variation is explained by the model.

Code

```

mae = mean_absolute_error(y_test_house, house_predictions)
rmse = np.sqrt(mean_squared_error(y_test_house,
house_predictions))
r2 = r2_score(y_test_house,
house_predictions)

regression_results =
    pd.DataFrame({ "Metric":
        ["MAE", "RMSE", "R2"],
        "Value": [mae, rmse, r2],
    })
regression_results

```

Output

	Metric	Value
0	MAE	53320.013050
1	RMSE	74558.138301
2	R2	0.575788

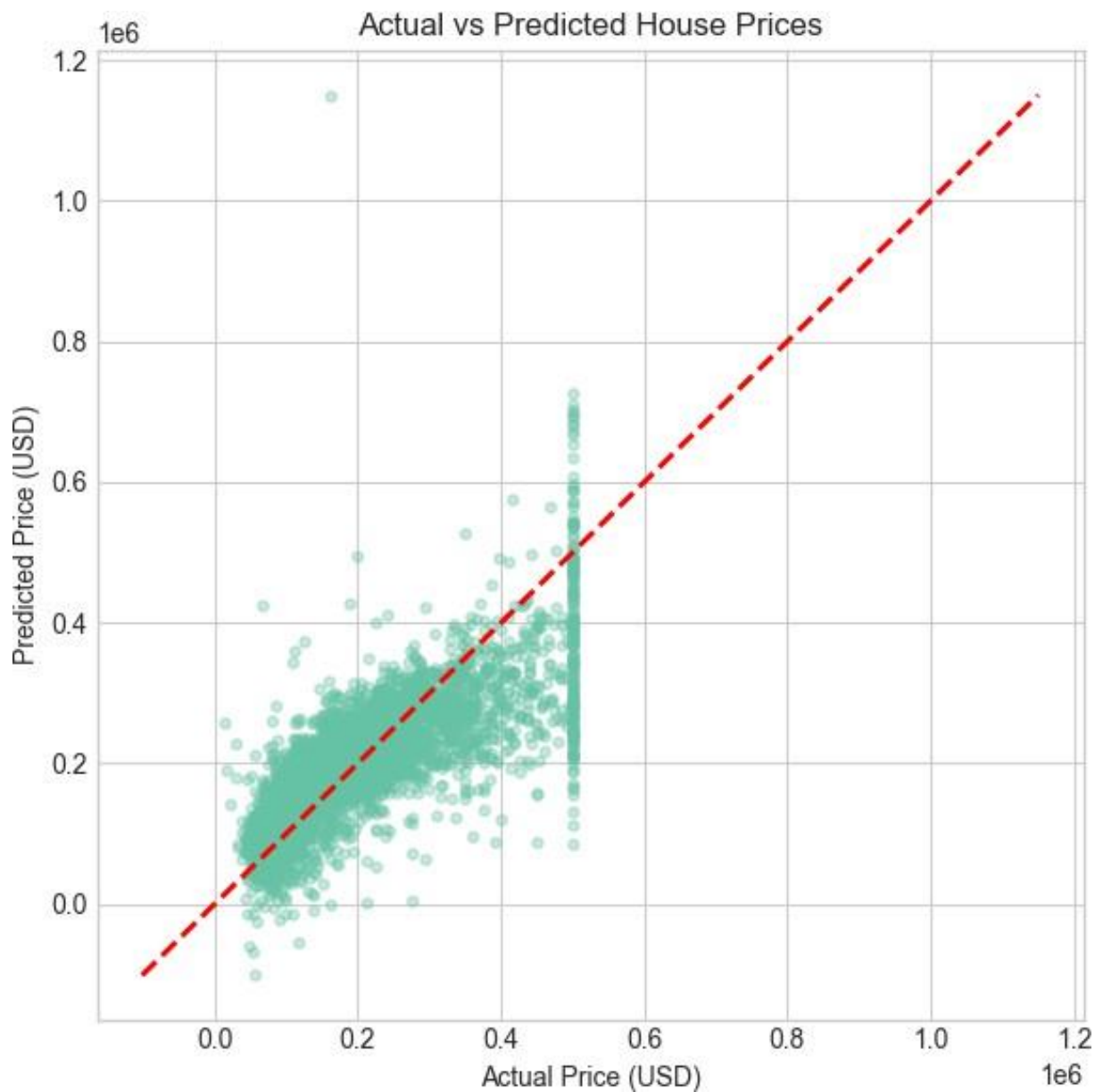
Code

```

fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(y_test_house, house_predictions, alpha=0.35,
s=15)
min_price = min(y_test_house.min(),
house_predictions.min())
max_price = max(y_test_house.max(), house_predictions.max())
ax.plot([min_price, max_price], [min_price, max_price], "r--", linewidth=2)
ax.set_title("Actual vs Predicted House Prices")
ax.set_xlabel("Actual Price (USD)")
ax.set_ylabel("Predicted Price (USD)")
plt.tight_layout()
plt.show()

print(f"Mean Absolute Error: ${mae:,.2f}")
print(f"Root Mean Squared Error:
${rmse:,.2f}")
print(f"R-squared: {r2:.3f}")

```



Output

Mean Absolute Error: \$53,320.01
Root Mean Squared Error:
\$74,558.14 R-squared: 0.576

Make Predictions on New Houses

The examples below represent new houses using the same feature columns used for training.

Code

```
new_houses = pd.DataFrame([
    {
        "MedInc": 4.5,
        "HouseAge": 20,
        "AveRooms": 5.5,
        "AveBedrms": 1.0,
        "Population": 900,
        "AveOccup": 3.0,
        "Latitude": 34.05,
        "Longitude": -118.25,
```

```

    },
    {
        "MedInc": 8.0,
        "HouseAge": 10,
        "AveRooms": 7.0,
        "AveBedrms": 1.1,
        "Population": 650,
        "AveOccup": 2.5,
        "Latitude": 37.77,
        "Longitude": -122.42,
    },
    {
        "MedInc": 2.0,
        "HouseAge": 35,
        "AveRooms": 4.0,
        "AveBedrms": 1.2,
        "Population": 1500,
        "AveOccup": 4.0,
        "Latitude": 36.75,
        "Longitude": -119.77,
    },
])

new_predictions = regression_model.predict(new_houses)
new_houses_with_predictions = new_houses.copy()
new_houses_with_predictions["PredictedPriceUSD"] =
new_predictions.round(2) new_houses_with_predictions

```

Output

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude
0	4.5		20	5.5	1.0		900
1	8.0		10	7.0	1.1		650
2	2.0		35	4.0	1.2		1500

	Longitude	PredictedPriceUSD
--	-----------	-------------------

Conclusion

Linear Regression learned a relationship between housing features and median house value. The model is useful as a baseline: it gives clear coefficients and fast predictions. Real banks and real estate firms use similar supervised-learning workflows, although they usually train more advanced models and validate them with stricter business rules.

